CAREBOT

Caregiver Connect — Crowdsourcing Team

Sprint 2 Presentation | CS 495 Senior Capstone | University of Alabama

Cole Segura

Carlos Marquez

Sheala Miller

Jaylon Sanders

Jarl Evanson

GitHub: github.com/Distrubuted-Autonomy-Lab/carebot-deployable-crowdsourcing

Website: https://distrubuted-autonomy-lab.github.io/crowdsourcing-senior-info-website/

March 2026

PROJECT FOCUS

What is Carebot and what are we building?

The Application

Django web app backed by PostgreSQL, PostGIS, and pgvector that helps Alabama residents find healthcare resources through a Google Gemini AI chatbot

Stack: Django 5.0, PostgreSQL 17, PostGIS, pgvector, Sentence Transformers, Gemini, Docker Compose

Sprint 2 Focus

- UI/UX Redesign: Accessible, grandma-friendly interface
- Vector Embedding Similarity: Full-catalog TF-IDF + pgvector deduplication
- Cloud Deployment: GCP Compute Engine demo
- Security Hardening: CSRF, Parameterized SQL, Safer logging

Our Focus

- Enable community resource submissions
- Build moderation workflow
- Implement merge/deduplication system
- Improve development environment & security

SPRINT 2 GOALS

Prioritized objectives for Feb 24 – Mar 24, 2026

UI/UX Redesign & Accessibility

Vector Embedding Similarity

Frontend Implementation (HTML/CSS)

Backend Integration (Django)

Testing & Documentation

MAJOR ACHIEVEMENTS

Backend & Testing

- Bug fixes and code cleanup across the backend
- App deployed to Google Cloud for live demo
- Test plan created for Sprint 3 execution

Frontend Build

- Login, home page, and navigation restyled
- Submission form reorganized into clear sections
- Responsive design for mobile and desktop

UI/UX Design

- Design spec covering all 5 crowdsourcing pages
- Accessibility guide for the team (WCAG 2.1 AA)
- Large touch targets, readable text, clear controls

Vector Embeddings

- New submissions are compared against the entire resource database for duplicates
- Uses two AI approaches together for more accurate matching
- Moderators can see similarity scores when reviewing submissions

PROJECT & SPRINT BACKLOG

Sprint 2 Backlog

ID	Item	Priority	Status
S2.1	Vector similarity (pgvector + full-catalog TF-IDF) & moderator dedup UX	P0	Done
S2.2	Security: embedding logs, parameterized SQL, security PRs	P1	In progress
S2.3	UI/UX design — layout, buttons vs. checks vs. dropdowns; no owl / out-of-scope chrome	P0	In progress
S2.4	Frontend — submission form & moderation queue HTML/CSS per UX spec	P0	In progress / merged
S2.5	Base + home + **login** (accessible patterns) — **user/staff login working**	P0	Done
S2.6	Django views, forms, logic for new UI & merge/submit	P0	In progress
S2.7	PR review & merge — integration checkpoint	P0	In progress
S2.8	Unit / integration tests — similarity & submission/moderation	P1	Update
S2.9	E2E, accessibility testing, QA	P1	Update
S2.10	Documentation — README, sprint docs, runbooks	P2	Updated
S2.11	GCP VM / cloud demo — Docker, firewall, secrets	P1	Done
S2.12	Scrum & stakeholder updates (embeddings + UX direction)	P2	In progress

Project Backlog

ID	Item	Priority
B1	Resource submission form + moderation queue	P0
B2	Approve, merge, and reject actions	P0
B3	Smart merge with deduplication	P0
B4	Similarity search for matching	P0
B5	Fuzzy matching improvements	P2
B6	Bulk moderation (multi-select)	P2

Non-functional Items

ID	Item	Priority
B8	Docker compatibility (Mac + Windows)	P1
B9	Security configuration improvements	P1
B10	Comprehensive end-to-end testing	P1
B11	Unit tests for submission & merge	P1
B12	Non-root Docker + HTTPS	P1
B13	Setup guides and documentation	P2

NOT COMPLETED & LESSONS LEARNED

Carrying to Sprint 3

- Execute test plan: unit, integration, and E2E tests
- Accessibility QA testing
- Bulk moderation (approve/reject multiple)
- Fuzzy matching improvements
- Full HTTPS + non-root production hardening
- TF-IDF performance optimization for large tables
- Automated re-embed after bulk import

Lessons Learned

- Design-first approach makes frontend implementation faster and more consistent
- Schema drift between SQL dumps and Django migrations caused local errors -- runbooks prevented repeated debugging
- Lexical + vector hybrid search is more robust than either alone
- Documenting Docker commands and schema fixes saved significant teammate onboarding time

Technologies

- Django 5.0, Python 3.12, PostgreSQL 17 + PostGIS
- pgvector, Sentence Transformers, scikit-learn TF-IDF
- Google Gemini API, Docker Compose
- GCP Compute Engine, GitHub
- HTML/CSS (WCAG 2.1 AA accessible)

TEAM CONTRIBUTIONS

Cole Segura

UI/UX design specs, accessibility guide, component decisions, presentation

Carlos Marquez

HTML/CSS implementation progress, form components built, login feature

Sheala Miller

Vector embedding work, security tasks, scrum coordination

Jaylon Sanders

Tests written, accessibility testing, documentation updates

Jarl Evanson

Django view/form updates, Cloud Deployment, PRs reviewed and merged

DEMO TIME